UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

CORSO DI INGEGNERIA DELLA CONOSCENZA

RELAZIONE CASO DI STUDIO

IL SOMMELIER ARTIFICIALE

Graziano Spinelli
Matricola 322084
g.spinelli10@studenti.uniba.it

<https://github.com/grazianospinelli/ICON2025>

AA 2024-25

Introduzione

Il progetto è dedicato al mondo del Vino, alla valutazione automatica del grado di bontà dei possibili abbinamenti che si possono effettuare tra Cibo e Vino ed alla possibilità di poter fare previsioni sul punteggio di valutazione assegnato ad un vino in base alle sue peculiarità.
Inoltre, l'idea di adattare il presente progetto al dominio della sommellerie nasce dall'opportunità del sottoscritto di poter interpretare contemporaneamente il ruolo di esperto del dominio, in quanto sommelier con diploma AIS e il ruolo di aspirante Ingegnere della Conoscenza.

Sommario

Il progetto si compone di due parti. Nella prima parte viene redatta una Base di Conoscenza contenente fatti sui principali piatti della cucina italiana e sui vini italiani ed internazionali con le loro caratteristiche organolettiche, assieme a delle clausole definite. Clausole utilizzate per asserire i principi di abbinamento tra cibo e vino secondo la metodologia Mercadini che prevede abbinamento per contraddizione e/o per concordanza sulla base di determinate caratteristiche del cibo e del vino.
L'utente potrà quindi fare query per richiedere di valutare l'abbinamento desiderato o più semplicemente per fare pattern matching per conoscere fatti sui vini o sui piatti di cucina: tipo quali sono i vini con determinate caratteristiche, ad es. i vini dolci o frizzanti o quali vini provengono da una determinata regione.
Nella seconda parte invece, tramite apprendimento supervisionato, si cercherà di risolvere un problema di regressione per imparare a predire il punteggio di un vino in base alle caratteristiche presenti nel dataset contemplato.

Elenco argomenti di interesse

- Rappresentazione della conoscenza in una Knowledge Base tramite clausole definite scritte in Logica del Primo Ordine (Prolog).
- Predizione del rating di un vino basato su algoritmi di apprendimento supervisionato per problema di regressione su target a valori continui, passando per
  - una fase di pre-processing delle features tra cui selezione delle feature più interessanti, one-hot encoding e rimozione di alcuni outlier.
  - model selection tra 4 modelli di regressori tramite k-Fold Cross Validation
  - una volta determinato il miglior modello viene effettuata la selezione dei migliori iperparametri per il modello tramite una griglia che testa 4x4x3x3x2 combinazioni di iperparametri per 10 cicli di Cross Validation.
  - Infine una volta determinati i migliori iperparametri per il modello si passa ad un ciclo finale di 30 Restart del modello con stati diversi + subsampling alla ricerca della migliore prestazione ottenuta che sarà la predizione finale.

Knowledge Base in Logica del Primo Ordine

Per implementare la base di conoscenza sono state inizialmente redatte 2 tabelle in formato Excel, una per i cibi ed una per i vini contenenti delle valutazioni in un range da 1 a 10 per le diverse caratteristiche organolettiche di entrambi. (1 bassa intensità – 10 massima intensità percepita)

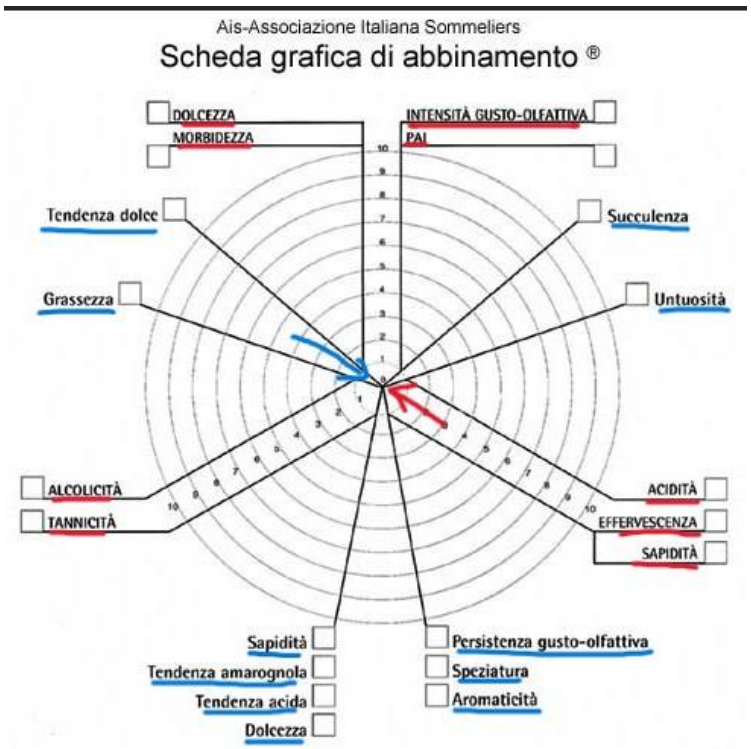
TABELLA VINI:

Vino	Regione / Paese	Tipologia	Effervescenza	Acidità	Sapidità	Alcoolicità	Tannicità	Morbidezza	Dolcezza	Persistenza
Barolo DOCG	Piemonte	Rosso	1	6	5	8	9	5	1	9
Barolo Riserva DOCG	Piemonte	Rosso	1	6	5	8	9	6	1	10
Barbaresco DOCG	Piemonte	Rosso	1	6	5	8	8	6	1	9
Barbaresco Riserva DOCG	Piemonte	Rosso	1	6	5	8	8	7	1	10
Barbera d'Asti DOCG	Piemonte	Rosso	1	8	6	7	5	6	1	7
Barbera d'Alba DOC	Piemonte	Rosso	1	8	6	7	5	6	1	7
Dolcetto d'Alba DOC	Piemonte	Rosso	1	5	4	6	7	5	1	6
Dolcetto di Dogliani DOCG	Piemonte	Rosso	1	5	4	6	7	5	1	6
Nebbiolo d'Alba DOC	Piemonte	Rosso	1	7	5	7	7	5	1	8
Gattinara DOCG	Piemonte	Rosso	1	7	5	8	8	5	1	9
Ghemme DOCG	Piemonte	Rosso	1	6	5	7	7	5	1	8

TABELLA CIBI:

Piatto	Categoria	Tend. Dolce	Grassezza	Untuosità	Succulenza	Speziatura	Sapidità	Tend. Amarognola	Tend. Acida	Dolcezza	Persistenza
Mozzarella di Bufala	Formaggio	2	6	6	5	1	5	2	3	2	5
Ricotta fresca	Formaggio	4	5	5	5	1	4	1	1	3	4
Taleggio	Formaggio	2	8	8	5	1	7	3	1	2	7
Pane di Altamura	Pane	1	2	2	3	1	5	2	2	1	5
Grissini Torinesi	Pane	1	3	3	2	1	6	2	1	1	4
Crescia Marchigiana	Pane	1	4	4	3	1	6	2	1	1	5
Piadina Romagnola	Pane	1	4	4	3	1	6	1	1	1	4
Pizza Margherita	Pizza	2	4	4	5	1	6	2	3	2	6
Pizza Marinara	Pizza	1	3	3	3	2	6	2	4	1	5
Pizza Quattro Stagioni	Pizza	2	5	5	5	2	7	2	3	1	6

L'idea base è quella di scrivere delle clausole definite in logica del primo ordine per rispondere a delle query su abbinamento tra cibo e vino secondo le regole del cosiddetto Metodo Mercadini. Un metodo che si basa sui principi di concordanza (similarità) o contrapposizione (contrasto) delle sensazioni sensoriali tra i due elementi esaminati, in cui ogni sensazione del cibo cerca un contraltare nel vino di intensità comparabile.



In particolare, in base allo schema di abbinamento Mercadini (rappresentato di fianco) si ha che le caratteristiche del cibo (evidenziate in blu) devono essere bilanciate dalle caratteristiche del vino (evidenziate in rosso) che si trovano nella parte opposta dello schema. I bilanciamenti possono essere per contrapposizione o per concordanza.

Abbinamenti per contrapposizione

Le sensazioni opposte si devono bilanciare per dare equilibrio in bocca:

- Durezza del cibo: sapidità, acidità o amarognolo contro morbidezza del vino (es. cibo salato/acido con vino morbido: consistente, passato in legno, affinato nel tempo).
- Grassezza del cibo contro sapidità o effervescenza del vino (es. formaggi freschi o salumi con vino spumante secco).
- Tendenza dolce del cibo (diversa dalla dolcezza, tipica dei carboidrati, crostacei, vegetali) contro acidità del vino. (es. preparazioni di mare con vini bianchi aciduli).
- Untuosità del cibo dovuta a olio o grassi fusi che si contrasta con il tannino del vino e la sua sensazione di astringenza. (es. sugo di brasato con vino rosso tannico come Barolo).
- Succulenza del cibo che si contrappone all'alcolicità del vino poiché l'alcool produce disidratazione in bocca. (es. Bistecca di Manzo con vino rosso alcolico come il Primitivo).

Questi abbinamenti si traducono nella seguente clausola Prolog:

```
contrapposizione(Cibo, Vino, Punteggio) :-
    piatto(Cibo, _, TdolcC, GrC, UntC, SuccC, _, SapC, TamarC, TacidC, _, _),
    vino(Vino, _, _, EffV, AcV, SapV, AlcoV, TannV, MorbV, _, _),

    % Contrapposizioni principali (durezza cibo vs morbidezza vino)
    ContrapDurezza is (10 - abs(MorbV - (SapC + TamarC + TacidC)/3 )),

    % Grassezza vs sapidita + effervescenza
    ContrapGrassezza is (10 - abs((SapV+EffV)/2 - GrC)),

    % Tendenza dolce vs acidità
    ContrapTendenzaDolce is (10 - abs(AcV - TdolcC)),

    % Untuosita vs tannini
    ContrapUntuosita is (10 - abs(TannV - UntC)),

    % Succulenza vs alcol
    ContrapSucculenza is (10 - abs(AlcoV - SuccC)),

    Punteggio is ContrapDurezza + ContrapGrassezza + ContrapUntuosita +
ContrapSucculenza + ContrapTendenzaDolce.
```

Nella Contrapposizione della Durezza viene fatta una media tra i valori di Sapidità, Tendenza Amarognola e Acidità del Cibo per bilanciare la Morbidezza del Vino, così come nella Contrapposizione della Grassezza dove viene fatta una media tra i valori di Sapidità ed Effervescenza del Vino. Il punteggio restituito dalla testa della clausola è pari alla somma delle 5 Contrapposizioni principali.

Abbinamenti per concordanza:

- Dolcezza del cibo con dolcezza del vino (es. dessert con vino passito o dolce).
- Intensità aromatica/speziatura del cibo con intensità aromatica del vino.
- Persistenza del cibo con PAI (persistenza aromatica intensa) del vino, intese come la durata delle sensazioni gusto-olfattive dopo la deglutizione. (Cibi persistenti come cacciagione con vini persistenti come l'Amarone)
- Struttura complessiva del cibo con corpo del vino. (Cibi strutturati con vini corposi)

Tali abbinamenti si traducono nella seguente clausola:

```
concordanza(Cibo, Vino, Punteggio) :-
    piatto(Cibo, _, _, GrC, _, _, SpezC, SapC, _, _, DolceC, PersC),
    vino(Vino, _, _, _, _, SapV, AlcoV, TannV, MorbV, DolceV, PersV),

    % Dolcezza cibo vs dolcezza vino
    ( (DolceV + DolceC) >= 14
        -> ConcDolcezza is DolceV + DolceC
        ;  ConcDolcezza is (10-abs(DolceV - DolceC))
    ),

    % Speziatura cibo vs intensita vino
    ConcSpez is (10 - abs((AlcoV + TannV + MorbV + PersV) / 4 - SpezC)),

    % Persistenza cibo vs Persistenza vino (PAI=Persistestanza Aromatica Intensa)
    ( (PersV + PersC) >= 14
        -> ConcPers is PersV + PersC
        ;  ConcPers is (10-abs(PersV - PersC))
    ),

    % Struttura cibo vs corpo vino
    ConcStruttura is (10 - abs((AlcoV+MorbV+TannV+PersV+SapV)/5 -
(SapC+SpezC+PersC+GrC+DolceC)/5)),

    Punteggio is ConcDolcezza + ConcSpez + ConcPers + ConcStruttura.
```

Dove l'intensità del Vino la ricaviamo dalla media tra Alcolicità, Tannicità, Morbidezza e Persistenza del vino che più sono alte e più il vino è intenso. Così come per il corpo del Vino o per la struttura del Cibo considerate come la summa delle loro caratteristiche principali.
Regole diverse invece sono state implementate nel corpo della clausola per calcolare la concordanza basata sulla dolcezza o sulla persistenza scegliendo di premiare questi 2 abbinamenti nel caso siano particolarmente azzeccati e quindi assegnando più punteggio nel caso in cui sia il Cibo che il Vino abbiano valori di dolcezza o di persistenza superiori o uguali a 7.
Il punteggio restituito dalla testa della clausola è pari alla somma delle 4 Concordanze principali.

Ancora una clausola è stata introdotta per calcolare la differenza tra il totale di tutte le caratteristiche del Cibo e il totale di tutte quelle del Vino intesa come differenza di armonia di abbinamento da sottrarre al calcolo finale (più armonioso sarà l'abbinamento e meno punti verranno sottratti al risultato finale).

```
diff_struttura(Cibo, Vino, Punteggio) :-
    piatto(Cibo, _, TdolcC, GrC, UntC, SuccC, SpezC, SapC, TamarC, TacidC,
DolceC, PersC),
    vino(Vino, _, _, EffV, AcV, SapV, AlcoV, TannV, MorbV, DolceV, PersV),
    Struttura_Cibo is TdolcC + GrC + UntC + SuccC + SpezC + SapC + TamarC +
TacidC + DolceC + PersC,
    Struttura_Vino is EffV + AcV + SapV + AlcoV + TannV + MorbV + DolceV + PersV,
    Punteggio is abs(Struttura_Cibo - Struttura_Vino).
```

La valutazione complessiva finale viene effettuata con le seguenti clausole che calcolano il punteggio finale come media tra il punteggio di Concordanza e quello di Contrapposizione a cui viene sottratto il punteggio di DisArmonia e che convertono poi tale punteggio in un grado di abbinamento.

```
%% ============================================================
%%  CLASSIFICAZIONE ABBINAMENTO
%% ============================================================

%% grado_abbinamento(Punteggio, Grado)
grado_abbinamento(P, 'eccellente')  :- P >= 38.
grado_abbinamento(P, 'buono')       :- P >= 34, P < 38.
grado_abbinamento(P, 'accettabile') :- P >= 30, P < 34.
grado_abbinamento(P, 'sconsigliato'):- P < 30.
```

```
%% ============================================================
%%  VALUTAZIONE COMPLESSIVA
%% ============================================================

% Valutazione complessiva (media 50% contrap, 50% concor) - Diff_Armonia
valutazione_dettagliata(Cibo, Vino, Contrap, Conc, Diff_Armon, Totale, Grado) :-
    contrapposizione(Cibo, Vino, Contrap),
    concordanza(Cibo, Vino, Conc),
    diff_struttura(Cibo, Vino, Diff_Armon),
    Totale is round(((Contrap + Conc) / 2) - Diff_Armon),
    grado_abbinamento(Totale, Grado).

valutazione(Cibo, Vino, Grado) :-
    valutazione_dettagliata(Cibo, Vino, _, _, _, _, Grado).
```

Ulteriori clausole che fanno semplice pattern matching sono state inserite per poter effettuare query per richiedere la regione di provenienza di un vino o tutti i vini di una determinata regione, oppure per richiedere la categoria di un piatto; per richiedere quali sono i vini dolci o quali sono quelli strutturati oppure quelli effervescenti, etc.

```
%% vino_per_tipologia
vino_per_tipologia(Tipologia, NomeVino) :-
    vino(NomeVino, _, Tipologia, _, _, _, _, _, _, _, _).

%% piatto_per_categoria
piatto_per_categoria(Categoria, NomePiatto) :-
    piatto(NomePiatto, Categoria, _, _, _, _, _, _, _, _, _, _).

%% piatto_speziato
piatto_speziato(Nome) :-
    piatto(Nome, _, _, _, _, _, Spez, _, _, _, _, _), Spez >= 6.

%% vino_strutturato (vino con alta persistenza e alcolicita)
vino_strutturato(Nome) :-
    vino(Nome, _, _, _, _, _, Alc, _, _, _, Pers),
    Alc >= 7, Pers >= 7.

%% vino_dolce
vino_dolce(Nome) :-
    vino(Nome, _, _, _, _, _, _, _, _, Dolc, _), Dolc >= 6.

%% vino_effervescente
vino_effervescente(Nome) :-
    vino(Nome, _, _, Efferv, _, _, _, _, _, _, _), Efferv >= 6.
```

La KB viene assemblata da un programma implementato in Python (**Make-KB.py**) che carica il contenuto delle tabelle Excel tramite metodi della libreria openpyxl, convertendo le informazioni relative al vino e al cibo in fatti per la KB. La Base di Conoscenza viene poi completata importando le clausole definite, presenti nel file testuale Regole_Abbinamento.txt.

È possibile testare il funzionamento del file generato KB.pl contenente i fatti e le regole sugli abbinamenti cibo/vino, ponendo query alla KB tramite interprete SWI-Prolog vers. 10.0.2 (implementazione open source del linguaggio di programmazione Prolog).
ESEMPI DI POSSIBILI QUERY:

```
?- vino_per_regione('Piemonte', NomeVino).
?- piatto_per_categoria('Contorno', Piatto).
?- vino_dolce(Nome).
?- valutazione_dettagliata('Parmigiano Reggiano', 'Chianti Classico DOCG',
Contrapposizione, Concordanza, Diff_Armonia, Totale, Grado_Abbinamento).
?- valutazione('Parmigiano Reggiano', 'Chianti Classico DOCG', Grado_Abbinamento).
```

ESEMPI DI QUERY EFFETTUATE IN AMBIENTE SWI-Prolog:

```
SWI-Prolog console
File  Settings  Tools  Debug  Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.1)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

101 ?- consult(['c:/Users/grazi/Desktop/ICON2026/datasets/kb.pl']).
true.

101 ?- valutazione('Parmigiano Reggiano', 'Barolo DOCG', Grado_Abbinamento).
Grado_Abbinamento = buono .

102 ?- valutazione('Pastiera Napoletana', 'Passito di Pantelleria DOC', Grado_Abbinamento).
Grado_Abbinamento = eccellente .

103 ?- valutazione('Orata alla Griglia', 'Passito di Pantelleria DOC', Grado_Abbinamento).
Grado_Abbinamento = sconsigliato.

104 ?- valutazione('Fonduta Valdostana', 'Franciacorta DOCG Brut', Grado_Abbinamento).
Grado_Abbinamento = buono .

105 ?- valutazione('Ossobuco alla Milanese', 'Barolo DOCG', Grado_Abbinamento).
Grado_Abbinamento = eccellente .

106 ?- valutazione('Triglie al Cartoccio', 'Locorotondo DOC', Grado_Abbinamento).
Grado_Abbinamento = accettabile .

107 ?- valutazione('Triglie al Cartoccio', 'Orvieto DOC Classico', Grado_Abbinamento).
Grado_Abbinamento = buono .
```

```
SWI-Prolog console
File  Settings  Tools  Debug  Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.1)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

101 ?- consult(['c:/Users/grazi/Desktop/ICON2026/datasets/kb.pl']).true.

102 ?- vino_per_regione(Regione,'Locorotondo DOC').
Regione = 'Puglia'.

103 ?- vino_per_regione('Puglia',Vino).
Vino = 'Primitivo di Manduria DOC' ;
Vino = 'Primitivo di Manduria Dolce Naturale DOCG' ;
Vino = 'Salice Salentino DOC' ;
Vino = 'Negramaro IGT Salento' ;
Vino = 'Nero di Troia DOC' ;
Vino = 'Castel del Monte DOCG Rosso' .

104 ?- vino_strutturato(Vino).
Vino = 'Barolo DOCG' ;
Vino = 'Barolo Riserva DOCG' ;
Vino = 'Barbaresco DOCG' .

105 ?- vino_dolce(Vino).
Vino = 'Moscato d\'Asti DOCG' ;
Vino = 'Asti Spumante DOCG' .

106 ?- piatto_grasso(Nome).
Nome = 'Lardo di Colonnata' ;
```

Apprendimento Supervisionato

La seconda parte del progetto verte sulla risoluzione di un problema di regressione, in particolare si vuole effettuare la predizione del punteggio di qualità che viene assegnato al vino in base alle sue caratteristiche principali come il prezzo, l'annata, la tipologia, il paese di produzione. L'obiettivo principale è far sì che l'algoritmo impari una relazione tra le feature di input e la variabile target **Rating** in modo che, una volta addestrato, possa fare previsioni accurate su nuovi dati mai visti prima.

Strumenti utilizzati

Il linguaggio utilizzato per lo sviluppo è stato il Python corredato dalle sue potenti librerie come Pandas per la gestione del dataset ed il relativo preprocessing, seaborn e matplotlib per la realizzazione dei grafici, kagglehub per il download del dataset da Kaggle ed infine scikit-learn con le sue API dedicate al Machine Learning.

Preprocessing del dataset

Il dataset inizialmente risulta decomposto in 4 files contenenti 4 macro-tipologie di vini: rosso, bianco, rosato e frizzante che vengono assemblati in unico file con l'aggiunta della colonna 'WineStyle'.

Name	Country	Region	Winery	Rating	NumberOfRatings	Price	Year	WineStyle
Pomerol 2011	France	Pomerol	Château La Providence	4.2	100	95	2011	red
Lirac 2017	France	Lirac	Château Mont-Redon	4.3	100	15.50	2017	red
Erta e China Rosso di Toscana 2015	Italy	Toscana	Renzo Masi	3.9	100	7.45	2015	red
Bardolino 2019	Italy	Bardolino	Cavalchina	3.5	100	8.72	2019	red
Ried Scheibner Pinot Noir 2016	Austria	Carnuntum	Markowitsch	3.9	100	29.15	2016	red

Alcuni dei vini riportati in tabella hanno il campo Year non specificato in quanto risultano essere assemblaggi di diverse annate. Poiché l'annata solitamente influisce sul prezzo e sul rating del vino si è preferito eliminare quelle righe del dataset che non contenevano annata.
Vengono eliminate anche le colonne Name, Region e Winery perché difficili da categorizzare, in quanto contenenti molti valori diversi, anche se solitamente l'importanza di una cantina o la regione di provenienza di un vino influiscono sul prezzo o sul rating del vino in modo consistente.
Dei restanti due campi categorici del dataset: Country e WineStyle, WineStyle ha 4 differenti valori e si presta bene ad una conversione one-hot encoding. Per quanto riguarda Country, il numero di nazioni diverse contenute nel dataset è 33, davvero al limite per effettuare un one-hot encoding ma si è deciso comunque di non perdere questa feature poiché come la regione di provenienza, anche la nazione di provenienza di un vino solitamente influisce sul suo prezzo.
Dopo la conversione, il numero totale di colonne del dataset diventa 41: Rating, NumberOfRatings, Price, Year + 4 colonne per WineStyle e 33 colonne per Country, per un totale di 13090 righe.
Poi, si passa alla rimozione degli OUTLIER individuati nella tipologia di vini con un punteggio molto basso ma che posseggono un prezzo che risulta nella media. Il numero totale di casi rimossi è 5.
Infine, viene definita la feature Target che sarà quindi la colonna Rating del dataset sulla quale andranno fatte le predizioni e si passa a suddividere le Features X dalla colonna Target:

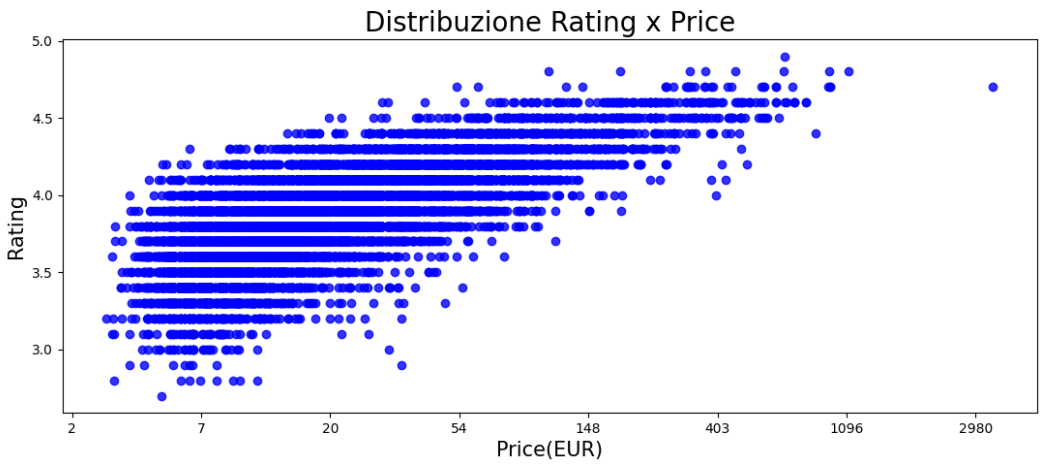
```
# Selezioniamo la feature target Y: Rating
y = wines.Rating

# Selezioniamo le Feature X
cols = [col for col in wines.columns if col != 'Rating']
X = wines[cols]
```

Visualizzazione delle Feature del dataset

Facciamo un grafico di tipo Heatmap per visualizzare le feature numeriche e la correlazione che c'è tra loro:

La correlazione tra Prezzo e Rating è la più forte: i vini più costosi tendono ad avere valutazioni più alte, però non è altissima; quindi, il prezzo non è l'unico fattore che determina la qualità del vino. Tra Rating e Anno c'è una correlazione negativa moderata: i vini più vecchi (anno più basso) tendono ad avere rating migliori, oppure i vini più recenti sono valutati peggio. Potrebbe indicare un effetto "invecchiamento" sulla qualità. Correlazione tra Prezzo e Anno negativa indica anche che i vini più vecchi tendono ad avere un prezzo più alto, il che è caratteristico del mondo del vino.
NumberOfRatings non sembra avere molta influenza sulle altre features.
Significativo anche il grafico che mette in relazione il prezzo del vino con il suo punteggio dal quale si evince che al crescere del prezzo cresce anche il rating. Si nota anche un Outlier con prezzo di 3000 euro circa ma valutazione simile ai vini di prezzo più basso che verrà rimosso dal dataset!

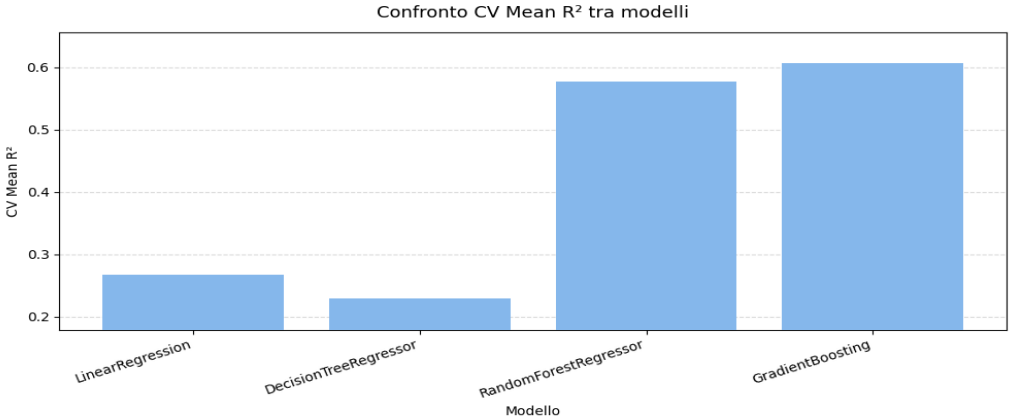


Model Selection

Sono stati scelti 4 modelli in grado di fare regressione da far competere tra loro con un set di iperparametri che inizialmente viene lasciato fissato:

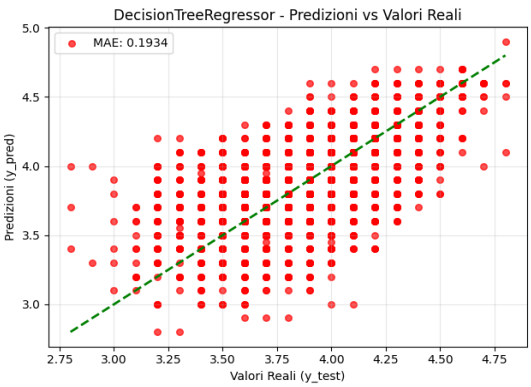
```
# Lista modelli da adattare
models = {
    'LinearRegression': LinearRegression(),
    'DecisionTreeRegressor': DecisionTreeRegressor(criterion='absolute_error', random_state=1),
    'RandomForestRegressor': RandomForestRegressor(n_estimators=150, random_state=1),
    'GradientBoosting': GradientBoostingRegressor(n_estimators=150, max_depth=6, min_samples_split=5, learning_rate=0.02, loss="squared_error", random_state=1)
}
```

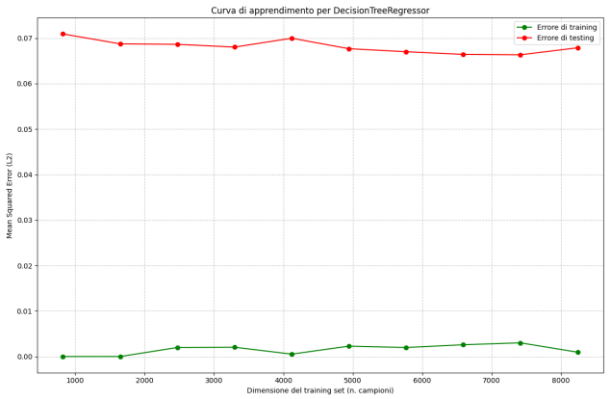
Il test viene effettuato all'interno di un for che cicla su tutti e 4 i modelli in cui viene utilizzata la funzione di scikit-learn denominata **cross_val_score** che consente di fare un numero prestabilito di iterate di K-Fold Cross Validation mantenendo nascosto ai vari modelli il test set ed operando sul solo training set che di volta in volta viene splittato in training e validation set.
Per ogni iterazione viene calcolato un punteggio di tipo R-quadro e alla fine, si ottiene un array di punteggi sul quale viene calcolata la media. La metrica R-quadro detta anche **coefficiente di determinazione** misura quanta variabilità dei dati reali il modello riesce a spiegare, su una scala da 0 a 1; è la metrica regina per valutare quanto un modello di regressione lineare si adatti bene ai dati e più è vicina ad 1 e migliore è il modello.
Dalla comparazione delle medie di R-quadro dei 4 modelli si ottiene il seguente grafico:



dal quale si evince che GradientBoosting Regressor è quello che performa meglio sul set di dati in oggetto. Inoltre, nel ciclo, per ogni modello viene realizzato un grafico Scatter che mostra quanto i valori predetti dal modello si discostano da quelli attuali e dalla predizione perfetta ed un grafico con la Curva di Apprendimento del modello al variare del numero dei campioni del training set.

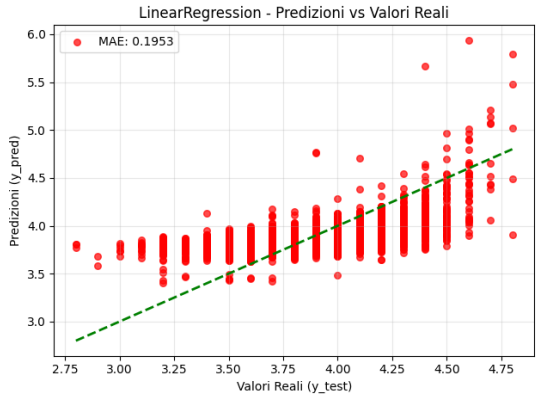
DECISION TREE REGRESSOR:

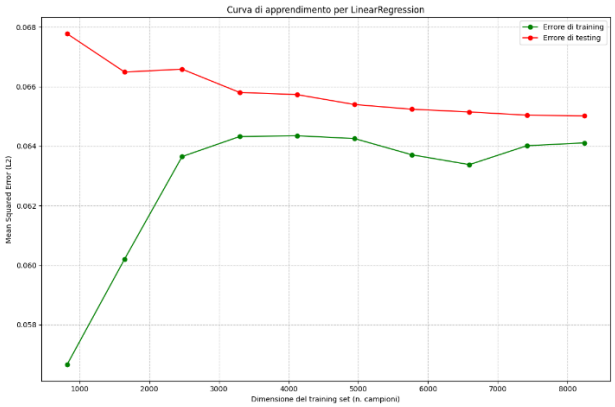




Il training error quasi zero indica che il modello memorizza molto bene i dati di training. Il test error attestato sul 0.07 risulta stabile ma molto lontano dal training ed il gap enorme tra le due curve indica un grave overfitting. Aggiungere dati non aiuterebbe perchè le curve non convergono.

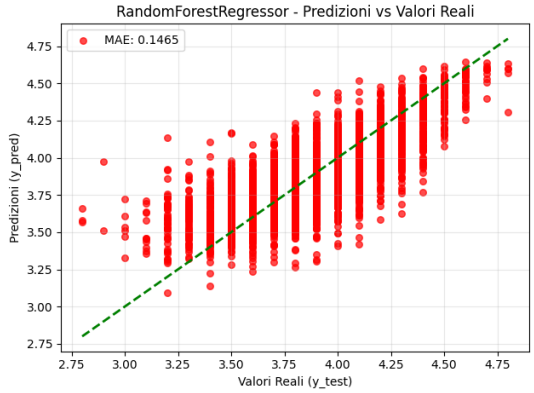
LINEAR REGRESSION:

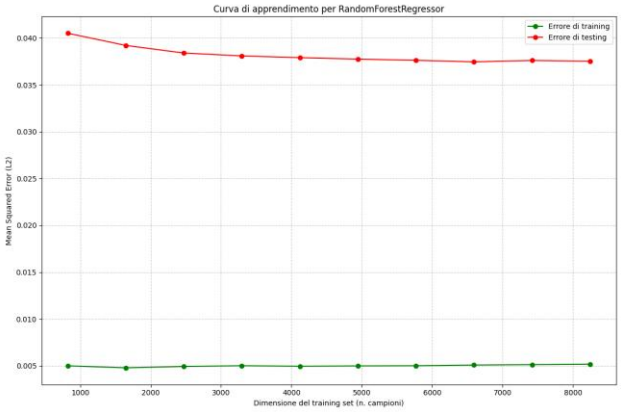




Training e test error sono molto vicini e questo implica poco overfitting. Le due curve si avvicinano, ma su valori di errore troppo alti. Aggiungere altri dati non servirà a molto perché il problema non è la quantità di dati, ma la capacità del modello. Il modello lineare sembra troppo semplice per catturare la complessità dei dati in quanto la curva di errore sul training è molto alta.

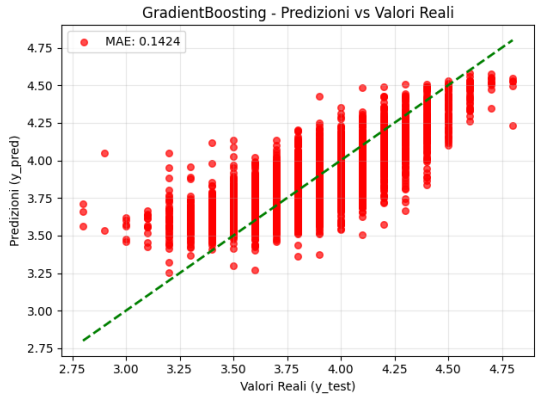
RANDOM FOREST REGRESSOR:

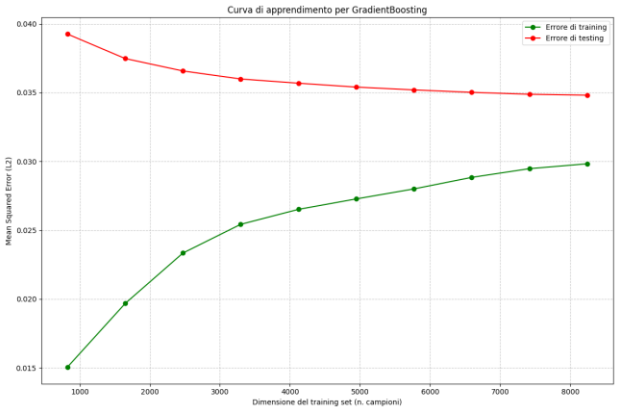




Training error quasi zero anche qui indica che il modello memorizza bene il training. Il test error è leggermente decrescente. Il gap è ampio ma minore rispetto al DecisionTree. C'è un overfitting moderato e più dati aiuterebbero leggermente, ma le curve non convergono.

GRADIENT BOOSTING REGRESSOR:





Il training error sale gradualmente da 0.015 a 0.030 all'aumentare dei campioni mentre il test error scende da 0.040 a 0.035. Le curve si avvicinano lentamente e questo è un buon segno, ma non convergono ancora. Aggiungere più dati potrebbe migliorare ulteriormente la situazione.

Iperparameters Selection

Una volta deciso il modello più performante, in questo caso il Gradient Boosting Regressor, si passa a fare il tuning degli iperparametri del modello. La strategia applicata per ricercare i migliori iper-parametri del modello è la **GridSearch con Cross Validation**. Tramite questa API di scikit-learn si definisce una griglia con un insieme di valori assegnabili agli iper-parametri del modello e si esplorano tutte le combinazioni possibili alla ricerca della miglior combinazione. Il totale dei tentativi fatti in questo caso specifico è pari 4x4x3x3x2 x 10 iterate di cross validation = 2880 iterate.

```
param_grid = {
    'n_estimators': [150, 200, 250, 300],
    'learning_rate': [0.05, 0.08, 0.1, 0.15],
    'max_depth': [2, 3, 4 ],
    'min_samples_split': [2, 3, 4],
    'min_samples_leaf': [1, 2]
}
```

Gli iperparametri ricercati sono così definiti:
- **n_estimators:** essendo il Gradient Boosting Regressor un modello di tipo ensemble, questo è il numero dei learner di base (alberi di decisione) utilizzati nel modello.
- **learning rate:** è il passo applicato nella ricerca del min della funzione Loss
- **max_depth:** Si riferisce alla profondità massima di ogni singolo albero di decisione che compone l'ensemble del Gradient Boosting.
- **min_sample_split:** stabilisce il numero minimo di campioni (osservazioni) che devono essere presenti in un nodo dell'albero per poter essere splittato.
- **min_sample_leaf:** è il numero minimo di campioni richiesti per installare un nodo foglia.

La funzione di Loss applicata nella ricerca è di tipo **L2 Quadratica.**

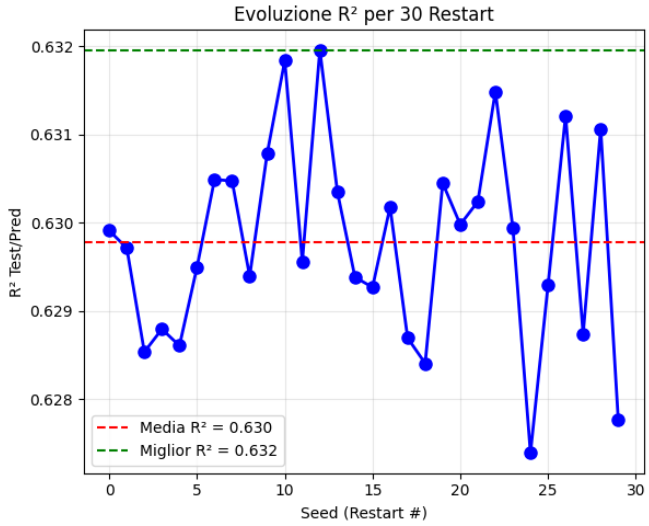
I migliori iperparametri selezionati per il Gradient Boosting Regressor dalla procedura sono:

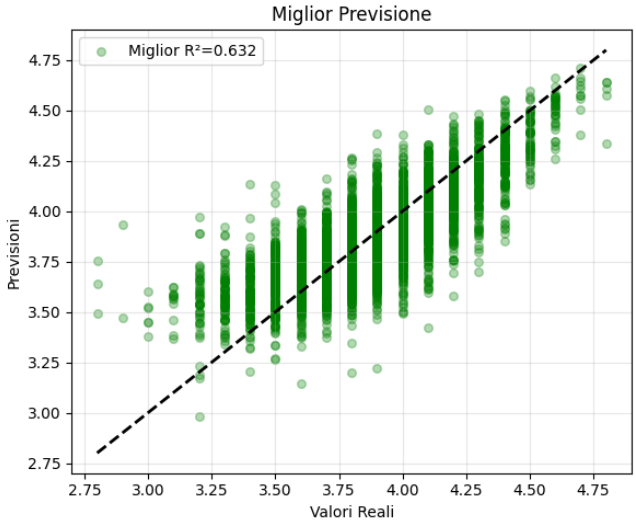
```
Migliori parametri: {'learning_rate': 0.1, 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 4, 'n_estimators': 300}
Miglior punteggio CV: 0.6212015599452968
```

Simil-Random-Restart Cycle

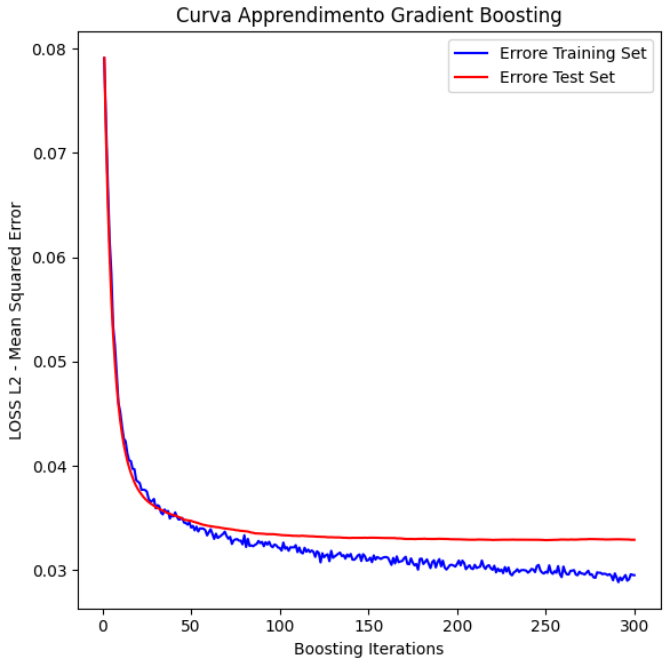
A conclusione della procedura si è pensato di applicare una procedura non convenzionale per la ricerca del miglior modello di predizione da applicare sul test set.
Invece di un tradizionale ciclo di diverse cross validation con split training/test set, si è implementato un ciclo in cui il Gradient Boosting, con i migliori iperparametri selezionati, viene fatto ripartire per 30 volte con un random_state diverso.
Il random state modificato in maniera incrementale ad ogni ciclo ed è utilizzato in combinazione con un altro parametro del Gradient Boosting denominato **subsample**, che settato con un numero minore di 1, attiva lo Stochastic Gradient Boosting che estrae casualmente ogni volta, un sottoinsieme diverso di dati dal dataset di training per effettuare il fitting, simulando una specie di Cross Validation nativa del modello, il cui effetto è comunque quello di ridurre l'overfitting.
Le predizioni di ogni ciclo vengono salvate in un dizionario con chiave pari alla metrica R-quadro calcolata sulla predizione stessa.

Alla fine, la predizione con R-quadro massima (0.632) viene selezionata come miglior predizione del modello.





A seguire la curva di apprendimento del Gradient Boosting con Stochastic Gradient Boosting attivato



Le differenze tra i valori attuali ed i valori predetti dal modello trovato:

```
RISULTATI:
R² medio singoli:   0.6298 ± 0.0011
R² migliore:        0.6320
       Attuali  Predetti
4148       4.3  4.235045
2817       4.0  3.750017
11101      4.2  4.167042
13055      3.6  3.612988
1124       3.9  3.727859
5149       4.1  4.075555
9467       4.3  4.423561
1273       4.0  3.859901
9029       3.7  3.747030
11158      3.9  3.851037
```

Conclusioni

Questo progetto ha dimostrato come sia possibile affrontare un dominio di conoscenza complesso e altamente specializzato come quello della sommellerie attraverso due approcci complementari propri dell'Ingegneria della Conoscenza: la rappresentazione esplicita della conoscenza tramite logica del primo ordine e l'apprendimento automatico supervisionato.

Nella prima parte, la costruzione della Knowledge Base in Prolog ha permesso di formalizzare in modo rigoroso le regole del Metodo Mercadini per l'abbinamento cibo-vino, traducendo in clausole definite principi che normalmente risiedono nella conoscenza tacita di un esperto sommelier. Il sistema risultante consente di rispondere a query articolate, valutare abbinamenti con un punteggio numerico e classificarli in gradi qualitativi (eccellente, buono, accettabile, sconsigliato), costituendo di fatto un sistema esperto nel dominio enogastronomico.

Nella seconda parte, la pipeline di apprendimento supervisionato ha permesso di selezionare il Gradient Boosting Regressor come modello più performante per la predizione del rating del vino, con un coefficiente di determinazione R^2 pari a 0.632. Tale valore, pur non essendo eccellente in termini assoluti, risulta soddisfacente per un fenomeno intrinsecamente soggettivo come il punteggio di un vino, che solitamente è influenzato da variabili difficilmente quantificabili come la reputazione della cantina, le condizioni climatiche dell'annata e le preferenze personali del recensore.

Tra i possibili sviluppi futuri del progetto si segnalano: l'arricchimento della Knowledge Base con un numero maggiore di vini e piatti, comprese cucine internazionali; l'integrazione di tecniche di Natural Language Processing per l'analisi delle descrizioni testuali dei vini, che potrebbero fornire feature aggiuntive utili alla predizione del rating; la realizzazione di un sistema di raccomandazione personalizzato che, combinando la KB e il modello predittivo, suggerisca all'utente il vino più adatto a un determinato piatto tenendo conto delle sue preferenze individuali.

Riferimenti Bibliografici

[1] Ragionamento logico: D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. 3e, Cambridge University Press [Ch.5]

[2] Prolog: D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. 3e, Cambridge University Press [Ch.15]

[3] Apprendimento supervisionato: D. Poole, A. Mackworth: Artificial Intelligence: Foundations of Computational Agents. 3e, Cambridge University Press [Ch.7]